

Answer Key for IA 01 - CPS 490-02

Andrew Golias

Part 1 - Read Repo State

1. HEAD is currently checked out branch

```
$ git symbolic-ref --short HEAD  
main
```

- HEAD represents a pointer that lists the currently checked out branch in the working tree. The command dereferences HEAD to return the symbolic reference name it targets, short removes the ref prefix which is the parent of the main branch. This command confirms HEAD is pointing to the main repository branch rather than a detached HEAD.

2. Branch reference identifies a commit

```
$ git rev-parse main  
d66359b0796ca45a7246b0a74a8c783988ed775b  
$ git cat-file -t main  
commit
```

- A git branch is not a file, but a reference identifying a specific commit object, which holds information about the changes made in the repository.
- Checking the object type of main proves that it is a commit, as opposed to a file containing the repository's objects.
- The commit object stores a blob that contains a snapshot of the state of the repository at the instance the branch is at.

3. Commit identifies tree & records parent relationship

```
$ git cat-file -p main  
tree 8a02ce7d2738ba507684f56930133a8d9380726f  
author Andrew Golias <goliasa1@udayton.edu> 1789079379 -0400  
committer Andrew Golias <goliasa1@udayton.edu> 1789079379 -0400
```

Initial commit

```
$ git cat-file -p 'main^{tree}'  
100644 blob 0a81a82254e3c3277344d189db4d91c8c7511562    README.md  
100755 blob f88995dbb5cef4cba28eb6e6d015668e8d525488    build-git-lab.sh  
100755 blob 875b14e400094c8a9a630cbc89ed38ac7ffc9a45    check-git-lab.sh  
100755 blob c920ca776ab5455622bdc48ac0430dea689bdfc2    reset-git-lab.sh
```

- The commit object stores its metadata and graph relations. The printed return is the snapshot of the root directory and the parent (previous commit). This proves a commit object points backwards to form a DAG. Creating a graph is the foundation of git interactions as it allows development to branch off of main development, restore history, and merge features.

4. Repository contains two nbranch tips that share earlier history

```
$ git graph
e5eeeee5 (HEAD -> main, origin/main) Reset git commits
c83414e Git build commit message
d66359b Initial commit
```

- git graph is my personal alias for `git log --graph --oneline --decorate --all`
- This command displays the repository's commit graph and all references. The output shows the initial commit to my repository with the pointer HEAD to main, as well as the state of the remote main branch (origin). With a more complex repository state, this command describes the branching structure and where features have been developed or merged.

Part 2 - Construct a Proposed Commit

```
$ git diff
diff --git a/README.md b/README.md
index 4ad011d..68e4c06 100644
--- a/README.md
+++ b/README.md
@@ -14,3 +14,7 @@ Application settings are stored in `config.properties`.
## Logging

Logging utilities are stored in `src/log.sh`.
+
+### Edits
+
+Entered README to make diff changes
diff --git a/config.properties b/config.properties
index f4c0461..f7f3d55 100644
--- a/config.properties
+++ b/config.properties
@@ -1,5 +1,5 @@
# Core runtime settings
-timeout=30
+timeout=60
retry.count=3
retry.delay.ms=250

@@ -7,10 +7,10 @@ retry.delay.ms=250
server.host=localhost
server.port=8080
connection.keepalive=true
-connection.pool.size=10
```

```
+connection.pool.size=20
```

```
# Logging settings
-log.level=INFO
+log.level=DEBUG
log.console=true
log.timestamps=true
log.file=app.log
:
```

- The working tree represents the uncommitted state of the local repo. This command shows differences in the changed files denoted by - and +. The - are the lines being removed from changes, + are the line being added in.
- When lines are being added, like in README.md, the changes are being directly added to the file. When an individual line is being changed, like in config.properties, the line is completely removed and the duplicated line with the new value is added in.
- The original file from the remote branch is labeled a/file_name, where the changed file is b/file_name

```
$ git add -p config.properties
diff --git a/config.properties b/config.properties
index f4c0461..f7f3d55 100644
--- a/config.properties
+++ b/config.properties
@@ -1,5 +1,5 @@
# Core runtime settings
-timeout=30
+timeout=60
retry.count=3
retry.delay.ms=250
```

```
(1/3) Stage this hunk [y,n,q,a,d,j,J,g,/,e,?]? y
```

```
@@ -7,10 +7,10 @@ retry.delay.ms=250
server.host=localhost
server.port=8080
connection.keepalive=true
-connection.pool.size=10
+connection.pool.size=20
```

```
# Logging settings
-log.level=INFO
+log.level=DEBUG
log.console=true
log.timestamps=true
log.file=app.log
```

```
(2/3) Stage this hunk [y,n,q,a,d,K,j,J,g,/,s,e,?]? n
```

```
@@ -21,6 +21,6 @@ feature.metrics=false
feature.experimental=false
```

```
# Limits
-max.connections=100
+max.connections=250
```

```
max.payload.kb=512
max.queue.depth=25
(3/3) Stage this hunk [y,n,q,a,d,K,g,/,e,?]? n
```

- Here, I staged the parts of the config.properties file. I only added the timeout update and left the two other hunks unchanged to be stored only locally. This will only push the one line when pushing file to the remote branch, meaning only one line change will be noted in the commit's snapshot.

```
$ git diff --staged
diff --git a/config.properties b/config.properties
index f4c0461..34f139a 100644
--- a/config.properties
+++ b/config.properties
@@ -1,5 +1,5 @@
 # Core runtime settings
-timeout=30
+timeout=60
 retry.count=3
 retry.delay.ms=250

$ git diff
diff --git a/README.md b/README.md
index 4ad011d..68e4c06 100644
--- a/README.md
+++ b/README.md
@@ -14,3 +14,7 @@ Application settings are stored in `config.properties`.
 ## Logging

 Logging utilities are stored in `src/log.sh`.
+
+### Edits
+
+Entered README to make diff changes
diff --git a/config.properties b/config.properties
index 34f139a..f7f3d55 100644
--- a/config.properties
+++ b/config.properties
@@ -7,10 +7,10 @@ retry.delay.ms=250
 server.host=localhost
 server.port=8080
 connection.keepalive=true
-connection.pool.size=10
+connection.pool.size=20

 # Logging settings
-log.level=INFO
+log.level=DEBUG
 log.console=true
 log.timestamps=true
 log.file=app.log
```

- Now when checking the differences in the staged file, the config file only shows the timeout change, because it is compared added files against the HEAD commit snapshot. When running `git diff`, the config file shows the changes on the two rejected lines but does not show timeout at all. This is because `--staged` specifies the snapshot that will be pushed to the remote, and without this tag, only local changes appear. This implies staged changes are no longer entirely 'local'

```
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   config.properties

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   README.md
    modified:   config.properties
```

- When checking the local status, config.properties appears as both a staged and unstaged file. There are lines changed locally that have not yet not prepared to send to the remote branch, and the timeout line that has been staged. This command does not check single file changes, rather it compares the snapshot against HEAD for both the staged and unstaged files and can duplicate them if individual lines are not added.

Part 3 - Explain Remote-Tracking State

```
$ echo "Alice feature addition" >> feature.txt
$ git add feature.txt
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   feature.txt

$ git commit -m "Alice local commit"
[main c2b7f89] Alice local commit
1 file changed, 1 insertion(+)
create mode 100644 feature.txt

$ git graph
c2b7f89 (HEAD -> main) Alice local commit
95b340d (origin/main) Update README
58b8886 Add logging
ba630c8 Add configuration loader
9ff682e Initialize project
```

- The `git graph` command proves Alice's feature is on a local branch `main`, while the remote repo is still behind on the latest `Update README` branch
- Only `03-remotes-alice/` is affected by this commit, `03-origin.git/` is unaffected until the commit is pushed and `03-remotes-bob/` is unaffected until pulled from the remote

```
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 295 bytes | 295.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
To /home/goliasa1/Documents/Capstone/ia-01-git-lab-student/git-lab-student/git-lab/03-origin.git
 95b340d..c2b7f89  main -> main

$ git graph
c2b7f89 (HEAD -> main, origin/main) Alice local commit
95b340d Update README
58b8886 Add logging
ba630c8 Add configuration loader
9ff682e Initialize project
```

- The push command transfers the new commit object to the remote repo, updating it's main branch pointer and changing Alice's local remote to reflect `origin/main`
- Now the `03-origin.git/` is updated by the push command and Bob's remote and pull changes

```
$ cd ../03-remotes-bob/
$ git graph
95b340d (HEAD -> main, origin/main) Update README
58b8886 Add logging
ba630c8 Add configuration loader
9ff682e Initialize project
```

- After switching to Bob's remote and running the graph command, it shows that Bob's `origin/main` does not point to the commit just made by Alice, which was pushed to the remote. This is because no changes have been made to Bob's remote that would update the tracking of the remote so it is still pointing to it's previous fetch.

```
$ git fetch origin
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 275 bytes | 275.00 KiB/s, done.
From /home/goliasa1/Documents/Capstone/ia-01-git-lab-student/git-lab-student/git-lab/03-origin
```

```
95b340d..c2b7f89  main      -> origin/main
```

```
$ git graph
c2b7f89 (origin/main) Alice local commit
95b340d (HEAD -> main) Update README
58b8886 Add logging
ba630c8 Add configuration loader
9ff682e Initialize project
```

- After fetching the remote and showing the graph, now it is shown that Bob is behind the remote and his HEAD is pointing to the same commit prior to fetching origin. Bob's local knowledge of the remote state is updated with this command, while not affecting his working state.
- This can allow Bob to be aware of remote changes made to pull those changes as he works and prevent merge conflicts down the line when his feature is completed and being integrated.

Part 4 - Compare Merge & Rebase

1. Merge

```
$ git graph
8a87ed6 (tag: D, feature/validation) Add validation tests
95b340d (HEAD -> main, tag: F) Update README
9ff682e (tag: A) Initialize project
ba630c8 (tag: B) Add configuration loader
ed1befd (tag: C) Add timeout validation
58b8886 (tag: E) Add logging
```

- Checking where branching occurred reveals that the remote is ahead of the local HEAD. This means that HEAD is attached to main at the commit with the tag, F. However, feature/validation points to the commit with tag, D. This proves both branches have acquired different commits since diverging from tag E.

```
$ git merge --no-edit feature/validation
Merge made by the 'ort' strategy.
 src/config.sh          | 10 ++++++++
 tests/test-timeout.sh | 10 ++++++++
 2 files changed, 20 insertions(+)
 create mode 100755 tests/test-timeout.sh

$ git graph
682dc03 (HEAD -> main) Merge branch 'feature/validation'
8a87ed6 (tag: D, feature/validation) Add validation tests
9ff682e (tag: A) Initialize project
ba630c8 (tag: B) Add configuration loader
ed1befd (tag: C) Add timeout validation
58b8886 (tag: E) Add logging
95b340d (tag: F) Update README
```

```
$ git cat-file -p HEAD
tree 29071292c2ff4a64c117e0ae89a6c3376c4e5
parent 6b111116d7b53c9e0cd3c61d018064f40268c406
parent 46befb4a25618b37e4fda1e5b4a07103b62c2acb
author Git Lab <git-lab@example.com> 1789682912 -0400
committer Git Lab <git-lab@example.com> 1789682912 -0400

Merge branch 'feature/validation'
```

- The merge command run on the main branch brings the feature branch into the working main branch. Reading HEAD proves, main has two parents (F and D) and it has diverged from its parents. Git creates a new merge commit branch snapshot that joins the histories without changing previous commit objects. This prepares the working branch HEAD to be in line with its remote when merging.

2. Rebase

```
$ git graph
46befb4 (HEAD -> feature/validation, tag: D) Add validation tests
6b11111 (tag: F, main) Update README
8baf4bd (tag: A) Initialize project
acc51ed (tag: B) Add configuration loader
b7d6ce6 (tag: C) Add timeout validation
490bf2d (tag: E) Add logging
```

Feature commit ID before base:

- feature/validation: 46befb4
- tag: D: 46befb4
- main: 6b11111

```
$ git rebase main
Successfully rebased and updated refs/heads/feature/validation.

$ git graph
0a2096f (HEAD -> feature/validation) Add validation tests
6fbcd8a Add timeout validation
6b11111 (tag: F, main) Update README
8baf4bd (tag: A) Initialize project
acc51ed (tag: B) Add configuration loader
b7d6ce6 (tag: C) Add timeout validation
46befb4 (tag: D) Add validation tests
490bf2d (tag: E) Add logging
```

Feature commit ID after base:

- feature/validation: 0a2096f
- tag: D: 46befb4

- main: 6b11111
- A rebase run on the feature branch detaches the feature commits (tag: C & D) and reproduces them such that main (tag: F) is pointing to them. The new parent of the divergence changes from tag: E to tag: F. Main's commit ID remains the same. Feature commits associated with tag: D receive a new ID. The commit for the feature/validation branch for which HEAD is pointing to receives a new ID.

Comparison

git merge preserves original branch history and existing commit object IDs by adding a single commit object with two parent references to document when merging occurred

git rebase recreates history by reproducing new features onto a new parent commit, creating a new commit object with a new ID to display a linear commit graph

Part 5 - Rewrite a Development History

```
$ git graph
78b95fb (HEAD -> feature/parser) remove debug output
076c99d fix test
1c24465 oops add debug helper
95faf65 Implement parser
6b11111 (main) Update README
490bf2d Add logging
acc51ed Add configuration loader
8baf4bd Initialize project
```

- There are 4 commits after main, the local HEAD is pointing to the branch feature/parser

```
$ git rebase -i HEAD~4
Successfully rebased and updated refs/heads/feature/parser.

$ git graph
c04b597 (HEAD -> feature/parser) Implement parser
6b11111 (main) Update README
490bf2d Add logging
acc51ed Add configuration loader
8baf4bd Initialize project
```

- When in the git editor window, to squash the 4 commits ahead of main onto a single commit snapshot, I left the first commit prefixed with pick. I then changed the next three commits to fixup, to combine them with the previous listed object.

```
$ bash ./tests/test-parser.sh
parser tests passed
```

- It is necessary to run a check after history rewrites because rebasing detaches commit objects to simplify the log. Rebasing can involve squashing, dropping, or reordering commits, which can accidentally delete lines or revert changes to past commits. Running a test after a rewrite confirms that the codebase is running as it is supposed to before the history is published.

Part 6 - Investigate and Resolve a Conflict

```
$ git show main:config.properties
# Core runtime settings
timeout=60
retry.count=3
retry.delay.ms=250

$ git show feature/conflict:config.properties
# Core runtime settings
timeout=DEFAULT_TIMEOUT
retry.count=3
retry.delay.ms=250

$ git show $(git merge main feature/conflict):config.properties
fatal: You have not concluded your merge (MERGE_HEAD exists).
Please, commit your changes before you merge.
# Core runtime settings
<<<<<< HEAD
timeout=60
=====
timeout=DEFAULT_TIMEOUT
>>>>>> feature/conflict
retry.count=3
retry.delay.ms=250
:
```

- Inspecting main shows the local timeout value of 60. The feature branch shows the value DEFAULT_TIMEOUT. Because both branches are off of a parent BASE, the last command shows the merge conflict of HEAD and feature/conflict. This conflict style shows a line-by-line difference between branches and allows developers to accept incoming changes or override them with current changes.

```
$ git merge feature/conflict
error: Your local changes to the following files would be overwritten by merge:
    config.properties
Merge with strategy ort failed.

$ git status
On branch main
Changes to be committed:
```

```
(use "git restore --staged <file>..." to unstage)
modified:   config.properties
```

- When attempting to merge feature/conflict into main, an error is returned explaining this conflict is preventing it. Git cannot infer which change is desired and aborts the merge putting this decision on the developer. The status of after the merge proves the config file is still staged and that the merge failed.

```
$ nano config.properties
$ git add config.properties
$ git merge --continue
[main 4f2ce0b] Merge branch 'feature/conflict'

$ head config.properties
# Core runtime settings
timeout=60
retry.count=3
retry.delay.ms=250

$ git status
On branch main
nothing to commit, working tree clean

$ git graph
4f2ce0b (HEAD -> main) Merge branch 'feature/conflict'
74c943b Part 7 - merge conflict
db96262 (feature/conflict) Use named default timeout
596ba6a Increase default timeout
6b11111 Update README
490bf2d Add logging
acc51ed Add configuration loader
8baf4bd Initialize project
```

- After editing the config file to remove the in-out changes from the conflict information, the file is added and the merge can be successfully completed. Checking the file, the correct value of 60 is listed for timeout. Checking the graph, main displays my commit message that feature/conflict has been merged.

Part 7 - Recover Apparently Lost History

```
$ git graph
14259cf (HEAD -> main) Add recovery marker
a4d1cb3 Enable demo mode
3e26571 Add recovery demo note
6b11111 (tag: F) Update README
490bf2d Add logging
acc51ed Add configuration loader
8baf4bd Initialize project
```

```
$ ls RECOVERY.txt
RECOVERY.TXT
```

The initial repo graph shows:

- tip of main: 14259cf
- commit tag F: 6b11111
- A recovery text file is present in this snapshot

```
$ git reset --hard F
HEAD is now at 6b11111 Update README

$ git graph
6b11111 (HEAD -> main, tag: F) Update README
490bf2d Add logging
acc51ed Add configuration loader
8baf4bd Initialize project

$ ls RECOVERY.TXT
ls: cannot access 'RECOVERY.TXT': No such file or directory
```

- Running a hard reset points the tip of main to tag F and its commit ID. The working tree now resembles commit F, and the commits after F are no longer listed in the graph. The recovery txt files on main earlier is no longer listed as a hard reset backrolls the state and eliminates any newly added files.

```
$ git reflog -5 --oneline
6b11111 (HEAD -> main, tag: F) HEAD@{0}: reset: moving to F
14259cf HEAD@{1}: commit: Add recovery marker
a4d1cb3 HEAD@{2}: commit: Enable demo mode
3e26571 HEAD@{3}: commit: Add recovery demo note
6b11111 (HEAD -> main, tag: F) HEAD@{4}: checkout: moving from main to main
```

- This command accesses the git logs without worrying about graph reachability, regardless of the reset. Once again is the 14259cf commit ID of main before the reset visible.

```
$ git branch rescue HEAD@{1}

$ git graph
14259cf (rescue) Add recovery marker
a4d1cb3 Enable demo mode
3e26571 Add recovery demo note
6b11111 (HEAD -> main, tag: F) Update README
490bf2d Add logging
acc51ed Add configuration loader
```

```
8baf4bd Initialize project
```

```
$ git checkout rescue
Switched to branch 'rescue'
```

```
$ ls RECOVERY.txt
RECOVERY.txt
```

```
$ cat RECOVERY.txt
This file exists so we can apparently lose it with reset --hard and recover
it
using the reflog.
```

- Rescuing the branch HEAD was initially on before the reset generates a graph similar to the one shown at the beginning of this part. When checking out this rescue branch, the RECOVERY.txt file can once again be read as it is only a part of this snapshot.
- Moving a branch reference simply updates the commit ID as a reference. It does not necessarily delete the commit or tree, thus the commit can be restored by running rescue. However, the branch is not automatically visible before rescuing it.

Part 8 - Search History w/ Bisect

```
$ git bisect start bisect-bad bisect-good
Bisecting: 4 revisions left to test after this (roughly 2 steps)
[49800ff46ffc0d6cc562df1957ba09afecd58b02] Refactor timeout normalization
```

```
$ git bisect run bash ./tests/test-compute.sh
running 'bash' './tests/test-compute.sh'
Bisecting: 2 revisions left to test after this (roughly 1 step)
[6b68e94d94d712856d8ddf2e2970abef1abb4106] Document pre-bug change 2
running 'bash' './tests/test-compute.sh'
Bisecting: 0 revisions left to test after this (roughly 1 step)
[986d27a532dafa9a2dff8aaae4633d222cc7d9ec] Document pre-bug change 4
running 'bash' './tests/test-compute.sh'
49800ff46ffc0d6cc562df1957ba09afecd58b02 is the first bad commit
commit 49800ff46ffc0d6cc562df1957ba09afecd58b02
Author: Git Lab <git-lab@example.com>
Date: Thu Sep 17 18:08:04 2026 -0400
```

```
Refactor timeout normalization
```

```
src/compute.sh | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
bisect found first bad commit
```

- Starting the bisect initiates a search across the repo graph between known good and bad endpoints. Running the bash script automates this search process by telling git to check out potential commits to evaluate each commit

```
$ git show 49800ff46ffc0d6cc562df1957ba09afecd58b02
commit 49800ff46ffc0d6cc562df1957ba09afecd58b02
Author: Git Lab <git-lab@example.com>
Date: Thu Sep 17 18:08:04 2026 -0400
```

Refactor timeout normalization

```
diff --git a/src/compute.sh b/src/compute.sh
index 84d9b60..30af51f 100755
--- a/src/compute.sh
+++ b/src/compute.sh
@@ -2,5 +2,5 @@

normalize_timeout() {
    local value="$1"
-   printf '%s\n' "$value"
+   printf '%s\n' "$((value + 1))"
}
```

- The bisect execution found a bad commit. When this is inspected, the specific line differences are shown (a changed print statement), which caused the test script to fail

```
$ git bisect reset
Previous HEAD position was 986d27a Document pre-bug change 4
Switched to branch 'demo/bisect'

$ git status
On branch demo/bisect
nothing to commit, working tree clean
```

- The bisect action is terminated and returns HEAD and the working directory to their original branch state from before the search, in this case demo/bisect.
- The script for bisecting evaluates commits based on their process exit code. A status 0 reflects a passing test, while a status 1 means it failed and that commit is bad. Git isolates the first bad commit to show where there failure occurs to allow restructuring at this point

Reflection

1. What is the difference between a commit and a branch?
 - A commit is an unchangeable snapshot of the repository at a specific instance in time, it stores an ID to be referenced at different points, a reference to its parent, and is an object storing changes made in that working session
 - A branch is a moveable divergence from the repository. Branches are used for adding features or testing, and identify a specific commit object.
2. What role does the index play in constructing the next commit?

- The index is a proposed snapshot for an upcoming commit. It is the middle state between the uncommitted local working tree and the committed snapshot referenced at HEAD. It allows a developer to stage files or lines of files to organize local changes without having to stage all working changes to the remote.
3. Why is origin/main local state, and what exactly does git fetch change?
- Origin/main is local because it is the most recently retrieved status of the remote from the network within the local working state. It must be local as otherwise would require live retrieval of the remote from the network, which is not always possible or wanted for working off a known functional state.
 - Git fetch changes this origin/main by updating the local cache of the state by connecting to the remote server upon request. It has no influence on the local branch reference or tree, only the state of origin/main with .git files.
4. How does a merge differ structurally from a rebase?
- A merge preserves the existing branch histories and commit IDs by creating a new merge commit object containing two parent pointers that link the divergent branch tips.
 - A rebase recreates history by detaching a branch's commit and placing its changes onto a new base commit object. This requires testing to ensure all changes are accurately recorded, while merging simply brings in changes from the working tree.
5. What information is Git missing when it reports a merge conflict?
- Git is missing the developer's intent when reporting a merge conflict. This would appear when two different branches make different changes on the same line, such as setting a variable to two different values. When these branches have the same parent (BASE), the (OURS) branch has different metadata than the (THEIRS) branch. Git is unable to process which is correct, it is only able to add/remove lines next to each other.
6. Why can the reflog sometimes recover history after a destructive-looking reset?
- `git reset --hard` is not fully destructive as it does not eliminate the commit object, rather it updates the branch reference pointer to an earlier commit. The reference log stores a local log of historical movement of HEAD. By looking through this log and rescuing a lost branch, git reveals the branch on the local graph to be used and read when the rescued branch is checked out.
7. What makes an automated test useful to git bisect?
- An automated test script signals success or failure of commits during the search. Git bisect runs a binary search across the commit graph to pick out a bad commit and this script aids in that process. The search model evaluates a large commit range in just a few steps and reduces the big-O notation of a traditional linear search.
8. What common mental model connects merge, rebase, reset, reflog, and bisect?
- The common mental model connecting these operations is a directed acyclic graph (DAG) of commit objects and moveable reference pointers. Git operations simply allow for navigation of this graph by joining graph paths (merge), reproducing nodes onto a new parent (rebase), moving pointers across nodes (reset), inspecting pointer movement history (reflog), and searching across a graph range

(bisect). These can all be applied to the graph model and make sense when understanding the concept of commits as objects and branches as references.